

Unified Inbox — System Design

Summary

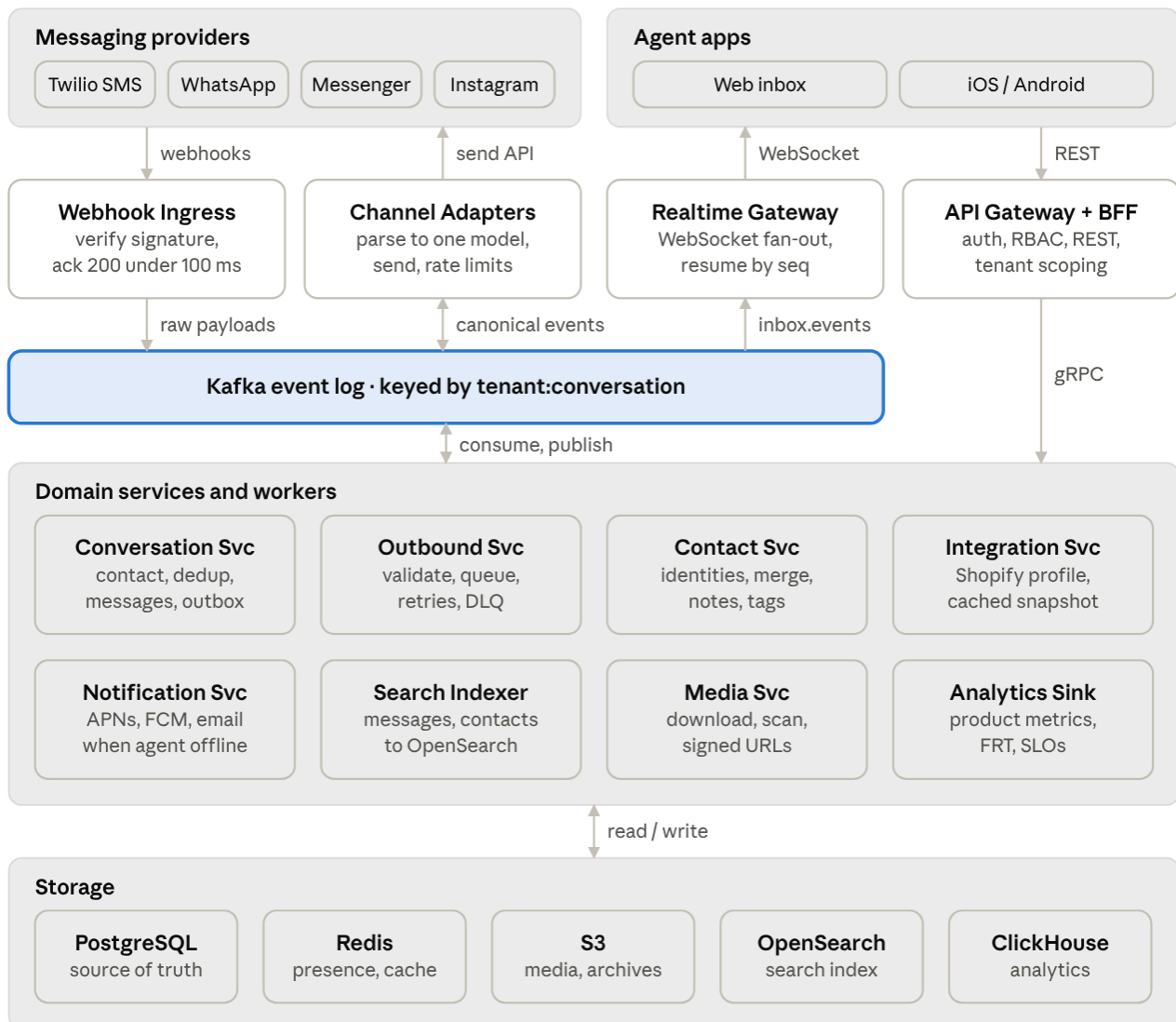
One adapter per channel, one Kafka log, PostgreSQL as the source of truth, WebSocket push to agents.

- **In scope:** send and receive on SMS, WhatsApp, Messenger, Instagram · unified contacts · assign, status, notes · real-time inbox · search
- **Out of scope (v1):** bots, campaigns, analytics UI. They can plug into the same event log later.

Assumption	Value
Tenants / agents	100k SMBs / 300k agents
Messages	20M per day, 2.5k/s peak
Inbound latency to agent screen	p95 < 1 s
Availability of send/receive	99.95%

High-level architecture

Every message flows through one Kafka log; PostgreSQL is the source of truth



High-level architecture · edge, event backbone, domain, storage

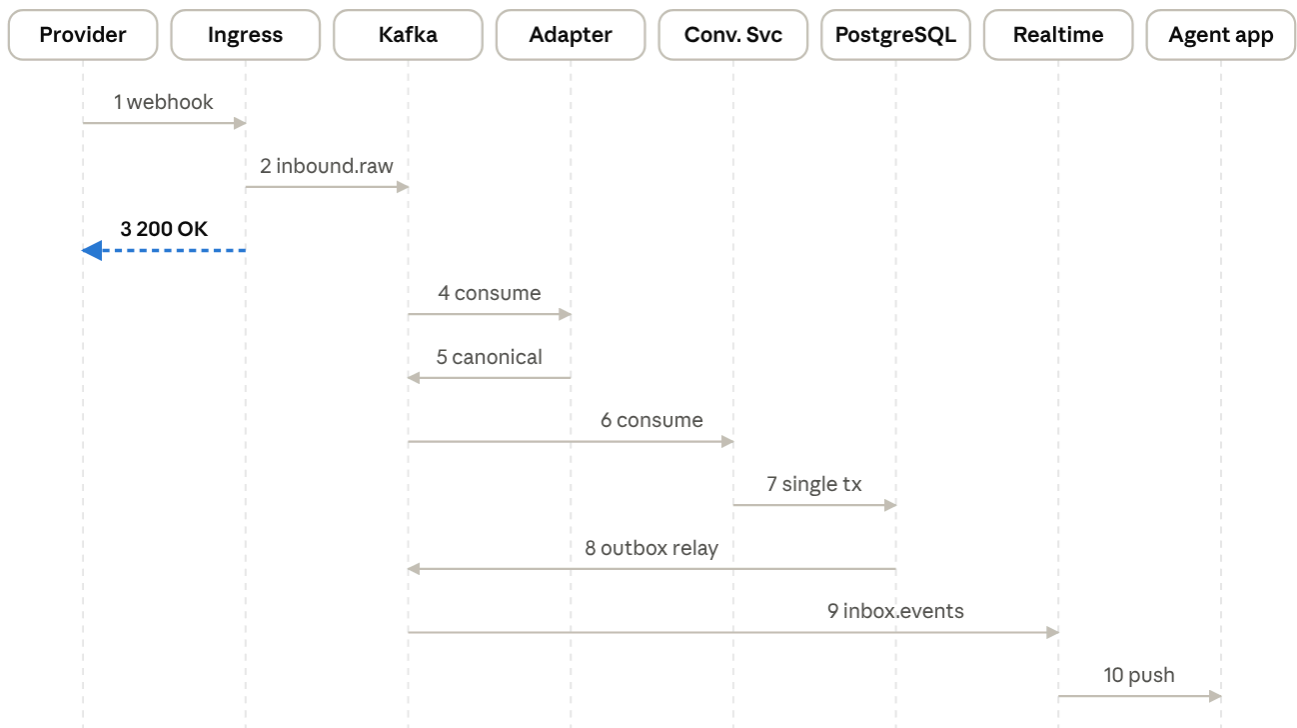
- Stateless Go services on Kubernetes; v1 ships as 3 deployables (edge, core, workers).
- A new channel = one new adapter behind **Parse / Send / Capabilities**. Nothing downstream changes.
- A slow provider never blocks the inbox: everything between edge and domain goes through Kafka.

Data flows and event schemas

Every message becomes one canonical **MessageEvent** on Kafka, keyed by **tenant:conversation**, so each conversation stays in order.

Inbound

Provider gets 200 after one Kafka write; everything else is async

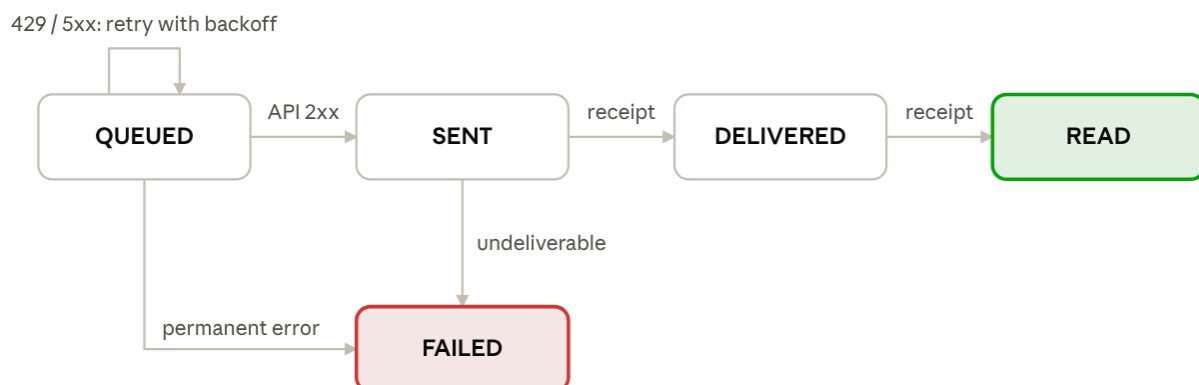


Inbound message sequence · 10 steps

Step 7 is one transaction: dedup, upsert contact and conversation, insert message, outbox row.

Outbound and delivery status

Message status only moves forward; failures are terminal



Outbound message states

- `POST .../messages` returns 202 with `QUEUED`; the adapter sends with a per-number rate limit.

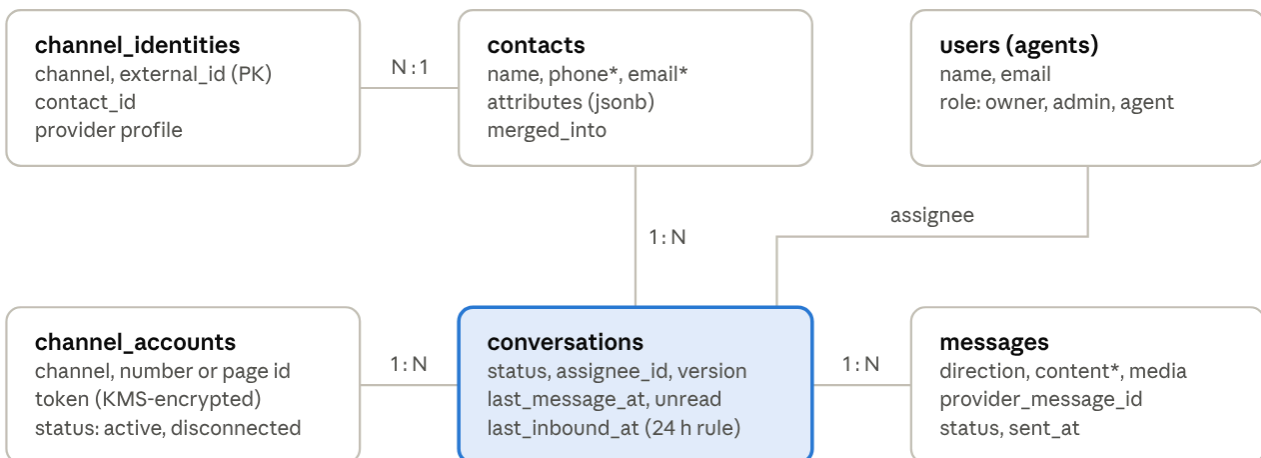
- Retries: 1 s, 5 s, 30 s, 2 min, 10 min, then dead-letter queue.
- Late receipts never move a status back (`READ` stays `READ`).

Canonical event

```
{
  "event_id": "01J9Z6Q8K3...",          // idempotency key
  "tenant_id": "t_4821",
  "channel": "WHATSAPP",
  "direction": "INBOUND",
  "external_user_id": "+14151235566",
  "provider_message_id": "wamid.HBgL...",
  "sent_at": "2026-09-30T12:40:03Z",
  "content": {"type": "TEXT", "text": "Hi, I have a question...", "media":
[]}]
}
```

Data model and storage

Conversation is the hub: one contact, one channel, many messages



* encrypted per tenant · every table keyed by tenant_id first · messages partitioned by month

Core data model (PostgreSQL)

Store	Holds	Main access pattern
PostgreSQL (sharded by tenant)	Everything above + outbox	Inbox list: keyset on (tenant, status, last_message_at); history: last 50 by (conversation, id)

Store	Holds	Main access pattern
Redis	Presence, unread counters, idempotency keys, rate limits	Key lookup, TTL
Kafka	Event log between services	Append, consume in order per conversation
S3	Media, raw webhooks	Pre-signed upload and download
OpenSearch	Search over messages and contacts	Full-text + filters, always by tenant

Writes go to the primary; inbox reads go to replicas. Dedup table on `(tenant, channel, provider_message_id)` makes webhook retries harmless.

API design

REST for commands and queries, one WebSocket for push. Cursor pagination everywhere; Idempotency-Key on every send.

Endpoint	Purpose
<code>GET /v1/conversations?status=&assignee=me&cursor=</code>	Inbox list
<code>PATCH /v1/conversations/{id}</code>	Assign, change status (<code>If-Match: version</code>)
<code>GET /v1/conversations/{id}/messages?before=</code>	History, 50 per page
<code>POST /v1/conversations/{id}/messages</code>	Send → 202 <code>QUEUED</code>
<code>POST /v1/conversations/{id}/notes</code>	Internal note
<code>GET/POST/PATCH /v1/contacts, POST .../merge</code>	Contacts
<code>GET /v1/search?q=</code>	Search
<code>POST /webhooks/{provider}/{account}</code>	Provider webhooks (signature-verified)
<code>WS /v1/realtime</code>	<code>message.created</code> , <code>message.status</code> , <code>conversation.updated</code> ; resume by <code>seq</code>

Privacy and security

- **Tenant isolation:** `tenant_id` only from the token; Postgres row-level security as a second guard.
- **Encryption:** TLS/mTLS in transit; per-tenant KMS keys for message bodies, phones, emails, provider tokens.
- **Webhooks:** verify every provider signature; reject replays older than 5 min.
- **No PII in logs:** ids only.
- **Compliance:** retention per tenant, GDPR delete/export, SMS STOP opt-out, WhatsApp 24 h rule enforced server-side, EU data residency.

Top metrics

North star: median first response time. It is why an SMB moves to messaging, and the product moves it directly.

Metric	Target
First response time (median, per tenant)	Goes down month over month
Conversations with no reply in 24 h	< 5%
Weekly active tenants, channels per tenant	Growing
Inbound latency, webhook to screen	p95 < 1 s
Outbound delivery rate (<code>delivered</code> / <code>sent</code>)	> 98% per channel
Kafka consumer lag	< 5 s

Scalability and reliability

Concern	Answer
Webhook never lost	Ingress acks only after a Kafka write (RF 3); if Kafka is down, return 503 and the provider retries
No duplicates	Dedup on provider id, <code>Idempotency-Key</code> on sends, idempotent consumers
No dual writes	Transactional outbox: the DB change and its event commit together

Concern	Answer
One provider down	Circuit breaker per channel; its sends queue, other channels keep flowing
Growth	Stateless pods autoscale on lag; Postgres shards by tenant; monthly message partitions
Noisy tenant	Per-tenant quotas on consumers and send rate
Region failure	Multi-AZ everywhere; async DR region (RPO < 1 min)

Edge cases

Case	Handling
Same webhook twice	Dedup, no-op
Out-of-order messages or receipts	Order by provider time; statuses only move forward
Same person on WhatsApp and SMS	Auto-link by phone; otherwise agent confirms a merge
Two agents reply at once	"Viewing" presence + optimistic lock on assignment
WhatsApp 24 h window closed	<code>WINDOW_CLOSED</code> ; UI offers approved templates
Customer texts STOP	Opt out; block sends until START
Provider token revoked	Channel <code>DISCONNECTED</code> , sends paused, owner notified
Media link expires	Download to S3 right away on inbound

Build order and open questions

1. **SMS end to end:** receive, reply, statuses, inbox, WebSocket.
2. **Meta channels:** WhatsApp, Messenger, Instagram.
3. **Workflow:** assignment, notes, push, contact merge.
4. **Search, Shopify, hardening:** load tests, DLQ tools, GDPR jobs.

Questions for the interview

- Are the capacity assumptions (100k tenants, 20M messages/day) close to the real target for the next 2 years?
- Do we use Meta's Cloud API directly, or a BSP such as Twilio for WhatsApp too?
- Should one contact have one conversation across all channels, or one per channel (current design)?
- Is multi-region data residency (EU) a launch requirement?
- Which integrations beyond Shopify matter first (HubSpot, Square, Salesforce)?